18.C21 Practice Midterm
Spring 2026 (Johnson & Marzouk) — Solutions

- The practice exam is likely to be both **harder** and **longer** than the real exam.
- You will have **50 minutes** after your exam begins.
- Write your **NAME** above *and* at the top of *every* page. Please do it as soon as you begin the exam, and check again before returning it.
- Only write on *one* side of each sheet. Additional paper will be provided upon request.
- You can bring **1 letter-sized sheet of notes** (both sides if you want), though it is unlikely to help much.
- **Please allocate your time carefully**, to make sure you get to work on all parts. Take a quick look at the *whole exam*, before starting working on your first problem.
- **Show your work / justify your answers.** Wrong answers may still receive partial credit *if* the reasoning was somewhat correct.
- If you absolutely believe there is something wrong with a problem, please ask. No hints, other than the ones already written in the exam, will be given.

Problem 1

Recall that double-precision (`Float64`) floating-point numbers store about 15–16 decimal significant digits, and can represent magnitudes from around $\approx 10^{-300}$ to around $\approx 10^{300}$ (after which they under/overflow, respectively).

1. Many libraries, including SciPy, provide a function $\text{logsumexp}(x)$ which takes a vector $x \in \mathbb{R}^n$ and computes the mathematical function:

$$\text{logsumexp}(x) = \log\left(\sum_{i=1}^{n} e^{x_i}\right).$$

   Explain why, for accuracy reasons (ignoring performance), this function should *not* be implemented naively using the above formula (e.g. *don't* do `log(sum(exp.(x)))` in Julia or `numpy.log(numpy.sum(numpy.exp(x)))` in Python). In fact, there are *two* extremes in which the naive formula goes horribly wrong and gives no correct digits in floating-point arithmetic. What are they? (Hint: think about $\max_i x_i$).

2. Outline (as pseudo-code) the steps of an modified algorithm to compute the same function without the catastrophes identified in the previous part. (Hint: factor something out of the sum, and remember the identity $\log(ab) = \log a + \log b$.)

Solution:

1. If $\max_i x_i$ is $\gg 1$, then e^x will **overflow** to `Inf`, the sum will give `Inf`, and then the log will give `Inf`. Conversely, if $\max_i x_i$ is $\ll -1$, then evern e^{x_i} will **underflow** to `0.0`, the sum will be zero, and log will either throw an exception or give `-Inf`, depending on the programming language. In either case you get nothing useful.

2. Let $x_k = \arg\max_i x_i$. Then simply factor this out of the sum

$$\begin{aligned}\text{logsumexp}(x) &= \log\left(e^{x_k}\sum_{i=1}^{n} e^{x_i - x_k}\right)\\ &= \log(e^{x_k}) + \log\left(\sum_{i=1}^{n} e^{x_i - x_k}\right)\\ &= x_k + \log\left(1 + \sum_{i\neq k} \underbrace{e^{x_i - x_k}}_{<1}\right).\end{aligned}$$

   This eliminates the over/underflow by taking out the biggest term x_k, and the remaining log term will be of an argument that is ≥ 1 but not particularly large, so it won't risk overflow.

   Remark: We can also use the `log1p` function provided by many numerical libraries, which computes $\text{log1p}(x) = \log(1 + x)$ accurately for small x, to evaluate the second log term even more accurately when the sum is very small, but that is a separate and much milder issue than the catastrophic over/underflow.

Problem 2

The "error" function $\mathrm{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2}\, dt$ is a standard special function (provided by many libraries, such as `scipy.special.erf`) that computes *part* of a Gaussian integral. It is plotted in Fig. 1 and has the property that $\mathrm{erf}(x) \to 1$ exponentially fast as $x \to \infty$, because $\int_0^\infty e^{-t^2}\, dt = \sqrt{\pi}/2$ (the famous "Gaussian integral").

1. The same libraries *also* provide a function $\mathrm{erfc}(x) = 1 - \mathrm{erf}(x)$, the "complementary error function." Why? Aren't erf and erfc redundant? What information would you potentially lose if you simply subtract erf from 1 yourself? Conversely, given erfc, why do you still need erf if you want don't want to lose information for some inputs, rather than using $\mathrm{erf}(x) = 1 - \mathrm{erfc}(x)$?

2. For large $x \gg 1$, the error function is approximately $\mathrm{erf}(x) \approx 1 - e^{-x^2}/(x\sqrt{\pi})$. Use this to compute the relative condition number $\kappa_r(\mathrm{erfc}, x)$ for large $x \gg 1$. And by inspection of the plot (of erf), what is the relative condition number of erfc (not erf) as $x \to 0$?

   Comment: These results explain why there should be an algorithm to compute erfc accurately when $\mathrm{erf}(x)$ is close to 1, despite the problem you identified with the naive formula in the previous part.

   (Recall that for any differentiable scalar function $f(x)$, $\kappa_r(f, x) = |f'(x)| \frac{|x|}{|f(x)|}$. Also recall the fundamental theorem of calculus, saying that the derivative is the inverse of integration: $(\int^x g(t)\, dt)' = g(x)$.)

3. erf is the integral of a smooth function e^{-t^2}, so you can compute it yourself for any given x very accurately with quadrature schemes like Clenshaw–Curtis (with errors

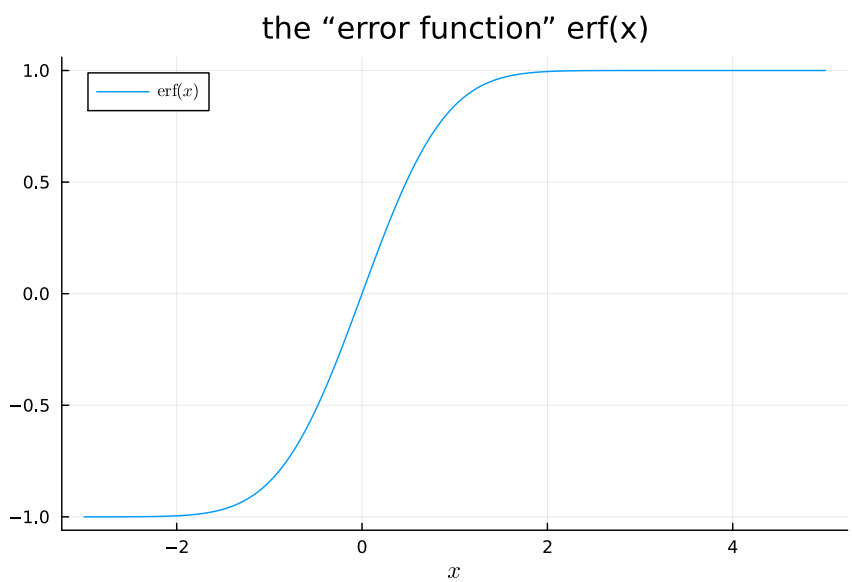

Figure 1: Plot of the error function $\mathrm{erf}(x)$.

that decrease at least exponentially fast with the number of quadrature points t). However, if we wanted to evaluate the function $\operatorname{erf}(x)$ for many different x values (and let's pretend you don't have a library function like `scipy.special.erf` available), say for $x \in [-10, 10]$, outline (as a sequence of steps) how you might construct an approximation $a(x)$ for $\operatorname{erf}(x)$ on $[-10, 10]$ such that (after the construction) the resulting $a(x)$ can then be evaluated quickly and accurately (to, say, at least 10 digits) for any given value of x, probably more quickly than re-doing quadrature for each x?

Solution:

1. The problem is that as $\operatorname{erf}(x) \to 1$, naively computing $1 - \operatorname{erf}(x)$ in floating-point arithmetic experiences catastrophic cancellation—we eventually lose *all* significant digits and just get `0.0` instead of a very small number. Conversely as $x \to 0$, we get $\operatorname{erf}(x) \to 0 \implies \operatorname{erfc}(x) \to 1$. This means that naively computing $1 - \operatorname{erfc}(x)$ would again lose *all* significant digits for $0 < |x| \ll 1$ and just give `0.0`, rather than a very small number. So, we need a separate erfc if we want to know how close erf gets to 1 for $x \gg 1$, while we need a separate erf if we want to know how small erf is for small $|x|$. We need both functions.

2. From the fundamental theorem of calculus, $\operatorname{erf}'(x) = \frac{2}{\sqrt{\pi}} e^{-x^2}$ (exactly!) and hence $\operatorname{erfc}'(x) = -\operatorname{erf}'(x) = -\frac{2}{\sqrt{\pi}} e^{-x^2}$. Furthermore, from the asymptotic formula given, we have $\operatorname{erfc}(x) \approx e^{-x^2}/(x\sqrt{\pi})$ for $x \gg 1$. Hence, plugging it in to the relative condition number formula, we get:

$$\kappa_r(\operatorname{erfc}, x) = |\operatorname{erfc}'(x)| \frac{|x|}{|\operatorname{erfc}(x)|} \approx 2x^2 \qquad \text{for } x \gg 1$$

   which grows with x but at least is not exponential!

   Conversely, as $x \to 0$, we can see from the plot that $\operatorname{erf}'(x) = -\operatorname{erfc}'(x)$ just goes to some nonzero constant, while $\operatorname{erfc}(x) = 1 - \operatorname{erf}(x) \to 1$. Hence

$$\kappa_r(\operatorname{erfc}, x) = |\operatorname{erfc}'(x)| \frac{|x|}{|\operatorname{erfc}(x)|} \to 0 \qquad \text{for } x \to 0$$

   because $|x| \to 0$ in the numerator and the other terms go to nonzero values.

3. One solution is simply to evaluate $\operatorname{erf}(x)$ at Chebyshev points on $x \in [-10, 10]$, construct a polynomial interpolant $a(x)$ (e.g. via the barycentric formula), and then use that. Then to evaluate it at any x we only need to evaluate the polynomial (which should be of fairly low degree since erf is smooth and hence polynomial approximation converges exponentially fast), faster than evaluating e^{-t^2} at a bunch of quadrature points

Problem 3

In Fig. 2(left) are shown three functions $f_1(x)$, $f_2(x)$, and $f_3(x)$ on $x \in [0, 1]$. On the right are shown the convergence of Clenshaw–Curtis quadrature to compute $\int_0^1 f_i(x)\,dx$ for $n+1$ point versus n for these three functions, labeled A, B, C, but we do *not* tell you which of f_1, f_2, f_3 are referred to by A, B, C.

1. Which function in Fig. 2(left) correspond to which convergence curve in Fig. 2(right), and why? Give a few words of justification, no formal analysis required.

2. For the two functions that are converging slowly in Fig. 2(right), explain how you might modify your integration algorithm (given a function that does Clenshaw–Curtis quadrature on any desired interval $[a, b]$ for any n) to get fast convergence for these two cases.

Solution:

1. From class, Clenshaw–Curtis is equivalent to a trapezoidal rule with a change of variables, and hence converges like a cosine series, and that the rate of convergence of a cosine series depends on how many derivatives are smooth. So the fastest convergence, convergence B, should correspond to f_1, which looks nice and smooth; the next-fastest convergence A should correspond to f_2 which has what looks like a discontinuous slope (a "kink") at $x = 0.75$; the slowest convergence C should correspond to f_3, which has a discontinuity, the most severe non-smoothness, at $x = 0.5$.

2. Simply break the integral into two smooth pieces. For f_2 you would compute $\int_0^1 f_2(x)dx = \int_0^{0.75} f_2(x)dx + \int_{0.75}^1 f_2(x)dx$, where each piece uses Clenshaw–Curtis separately. Similarly, for f_3 you would split the integral at $x = 0.5$. In both cases you would then have two smooth pieces with $\approx n/2$ points each (say), each of which gives quadrature that converges exponentially with $n/2$, and hence an overall error that converges exponentially with n (similar to convergence B for f_1).

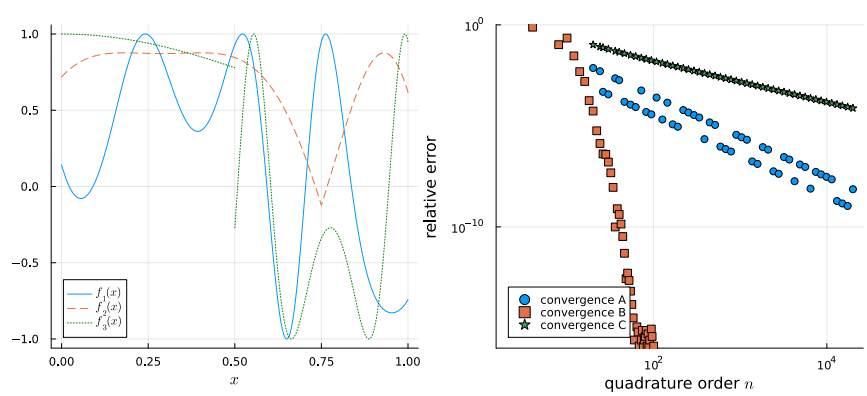

Figure 2: Plot of three functions $f_1(x)$, $f_2(x)$, and $f_3(x)$ on $x \in [0, 1]$ (left), and the convergence versus Clenshaw–Curtis quadrature order n ($n+1$ points) for these three functions, but in an unspecified order.

Problem 4

In problem set 1, you found that the quadratic curve $q(x)$ that goes through the three points $(0, f(0))$ and $(\pm\Delta x, f(\pm\Delta x))$ is:

$$q(x) = \frac{1}{2\Delta x^2}\left[f(-\Delta x)x(x-\Delta x) - 2f(0)(x^2 - \Delta x^2) + f(+\Delta x)x(x+\Delta x)\right]$$

Explain how you would use this to **derive a quadrature rule** to estimate the integral

$$\int_{-\Delta x}^{+\Delta x} f(x)\,dx \approx w_1 f(-\Delta x) + w_2 f(0) + w_3 f(+\Delta x)\,,$$

similar to how we derived the trapezoidal rule in class. **Give the weights** w_1, w_2, w_3. (This is called "Simpson's rule.")

Solution: We can estimate the integral by the integral of $q(x)$, just as for the trapezoidal rule we estimated the integral by integrating a piecewise-linear interpolant. So, the three weights are just the integrals of the coefficients:

$$w_2 = -\frac{1}{\Delta x^2}\int_{-\Delta x}^{+\Delta x}(x^2 - \Delta x^2)dx = -\frac{1}{\Delta x^2}\left(\frac{x^3}{3} - x\Delta x^2\right)\Bigg|_{-\Delta x}^{+\Delta x} = \boxed{\frac{4}{3}\Delta x}\,,$$

along with

$$w_3 = \frac{1}{2\Delta x^2}\int_{-\Delta x}^{+\Delta x}(x^2 + x\Delta x)dx = \frac{1}{2\Delta x^2}\left(\frac{x^3}{3} + \frac{x^2}{2}\Delta x^2\right)\Bigg|_{-\Delta x}^{+\Delta x} = \boxed{\frac{1}{3}\Delta x}\,,$$

with

$$w_1 = \boxed{w_2}$$

by symmetry (the two integrands are mirror images).

Problem 5

Suppose that we have 100 measurements (p_k, v_k) of the volume v of a gas vs. its pressure p, and we want to fit it to a function of the form $v(p) = \frac{c_1}{p} + c_2$ for unknown constants c_1, c_2. Write down a 2×2 system of equations you could solve to find c_1, c_2 in order to minimize the sum of the squared errors $\sum_k [v(p_k) - v_k]^2$. You can write your answer (left- and right-hand sides) as products of matrices and/or vectors, as long as you specify what each term is (in terms of the unknowns c_1, c_2 and/or the data $p_1, \ldots, p_{100}$ and $v_1, \ldots, v_{100}$).

(Recall that for least-square problems $\min_x \|Ax - b\|_2$ we derived the normal equations $A^T Ax = A^T b$ for the minimizer x, though we will see that there are often better ways to solve such problems.)

Solution: This is a least-square problem, so the answer is to solve the normal equations $\boxed{A^T Ac = A^T b}$ for $c = \begin{pmatrix} c_1 & c_2 \end{pmatrix}^T$ where

$$\boxed{A = \begin{pmatrix} \frac{1}{p_1} & 1 \\ \frac{1}{p_2} & 1 \\ \vdots & \vdots \\ \frac{1}{p_{100}} & 1 \end{pmatrix}} \text{ and } \boxed{b = \begin{pmatrix} v_1 \\ v_2 \\ \vdots \\ v_{100} \end{pmatrix}}$$

so that Ac is the "model" $\begin{pmatrix} v(p_1) \\ v(p_2) \\ \vdots \\ v(p_{100}) \end{pmatrix}$ and b are the data we are fitting to, so that $\sum_k [v(p_k) - v_k]^2 = \|Ac - b\|^2$.

Problem 6

The $\operatorname{sinc}(x)$ function is defined by $\operatorname{sinc}(x) = \frac{\sin(\pi x)}{\pi x}$ and shows up in many important applications in signal-processing and elsewhere. It is plotted in Fig. 3, below, along with the numerator $\sin(\pi x)$ for comparison.

While the $\sin(\pi x)$ function has extrema (minima/maxima) at $x = 0.5, 1.5, 2.5, \ldots$, the extrema of the sinc function are slightly shifted—for example, the first minimum, and in fact the global minimum, of sinc is at $x \approx 1.43$, shown by a black dot in the figure.

Outline the steps of an efficient algorithm to rapidly compute $\min_x \operatorname{sinc}(x) \approx \operatorname{sinc}(1.43)$ to very high accuracy (say, more than 12 digits), making use (somehow) of Newton steps (recall: to solve $f(x) = 0$ we take steps $x \to x - f'(x)^{-1} f(x)$).

Solution: Extrema occur when $\operatorname{sinc}'(x) = 0$, for which the Newton steps are:

$$\boxed{x_{k+1} = x_k - \frac{\operatorname{sinc}'(x_k)}{\operatorname{sinc}''(x_k)}},$$

starting at a good guess, say the $x_1 = 1.43$ value provided by the problem. Or, if we didn't know that approximation, we could simply use $x_1 = 1.5$ from the first minimum of $\sin(\pi x)$.

If we do this numerically to arbitrary precision (which obviously you cannot do on an exam!), we find that iterating from $x_1 = 1.43$ we have obtain $x_2 = \mathbf{1.43029}59167\ldots$, $x_3 = \mathbf{1.4302966531242}001\ldots$, $x_4 = \mathbf{1.430296653124202757772198445453}675\ldots$, and so forth, where the correct digits are shown in bold. So we obtain 15 significant digits with x_3, in only two steps, thanks to the quadratic convergence of Newton!

The necessary derivatives can be easily computed analytically: $\boxed{\operatorname{sinc}'(x) = \cos(\pi x)/x - \operatorname{sinc}(x)/x}$ (called "$\operatorname{cosc}(x)$" in Julia) and $\boxed{\operatorname{sinc}''(x) = -\pi^2 \operatorname{sinc}(x) - 2\cos(\pi x)/x^2 + 2\operatorname{sinc}(x)/x^2}$.

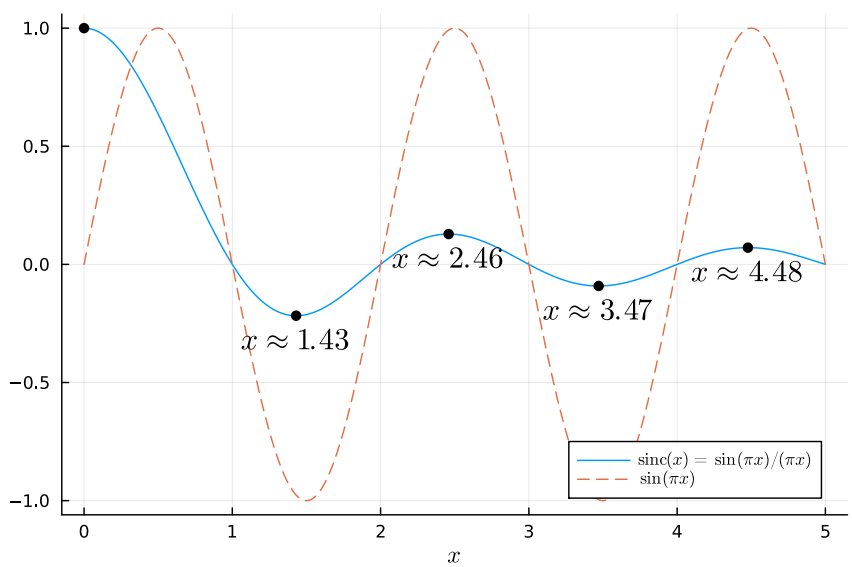

Figure 3: Plot of $\operatorname{sinc}(x)$, along with its numerator $\sin(\pi x)$. Extrema of the sinc function are indicated by labelled black dots.